Prompt Documentation

Clinic Enquiry Desk — Task 1: AI Skills & Prompt Engineering Assessment (AI Growth & Automation Associate)

Candidate: Srajan Shetty

Project: Clinic Enquiry Desk — an AI-built triage prototype for GP clinic enquiries (healthcare domain, Automation / Innovation area). It uses synthetic data only and gives no medical advice. A human approves every reply and emergency wording always wins.

Dates (IST): 30 October 2026 (build and upgrade rounds), 31 October 2026 (this document)

1. How to read this document

- Section 2 is an overview of every prompt, in the order I used them, with what each one was for.
- Section 3 shows how my prompting changed from one round to the next.
- Section 4 lists the messages I sent to Claude to plan the work and get the prompts written.
- Section 5 is the **full, unedited text of each prompt** sent to the AI coding tools.
- Section 6 states honestly what this record does and does not contain.

Prompt-writing support: I used **Claude** (claude.ai) to plan the project and to draft the long prompts from my instructions. I then ran those prompts in AI coding tools, read the results, and sent my feedback back to get improved prompts. AI responses are not reproduced here; only prompts are.

2. Overview of all prompts

#	Requested (IST)	Tool	Purpose	What happened
1	30 Oct 2026, 14:22	Codex	**Initial build prompt.** Build the whole project from scratch in one run: a GP-clinic enquiry triage prototype with an explainable classifier, one shared rules file, Python tools, synthetic data with labels kept separate, tests, safety notes and submission documents.	Codex produced the first working Clinic Enquiry Desk (Node server app with explainable triage, emergency flags, editable reply drafts, explicit approval and a hash-linked activity log). Git commits were blocked in its environment.
2	30 Oct 2026, 14:33	Codex	**Upgrade prompt (realistic and advanced).** Follow-up after reviewing the first build: audit it against the original plan, then add dual mode (static demo plus server), roles and routing, business hours and SLA timers, sensitive-data masking, duplicate detection, a tamper-evident log, an optional safe AI second opinion, a larger evaluation, security and accessibility work, and CI.	Run in Codex until its free token allowance was nearly used up. The remaining gaps were found and finished with prompt 4.
3	30 Oct 2026,	Codex	**Handoff request (token limit).** Codex was running low on tokens, so I asked it	Codex's login expired before this could be used (error shown at 14:47), so

	14:40		to write a handoff file that another AI tool could use to continue from exactly where it stopped.	prompt 4 was used instead.
4	30 Oct 2026, 14:47	Second AI coding tool (its output footer shows the model name “Big Pickle”)	**Continue-without-notes prompt.** Let a different tool inspect the half-finished folder itself, write a state file (done / unverified / missing) and finish the work in a fixed priority order.	It added the static GitHub-Pages-friendly demo mode, one rules.json shared by the JavaScript and Python classifiers, explainable match output, emergency recall in the evaluation (4 of 4 emergency cases), role routing, SLA timers, sensitive-data masking, Verify integrity and Simulate clinic day. It reported npm test 6/6 and Python unittest 12/12 passing, and fixed git by adding a safe.directory exception.
5	30 Oct 2026, 15:14	AI coding tool (Codex or the second tool)	**Wow-factor design prompt.** The working app still looked like a plain form-and-results page, so I asked for a distinctive design system, an animated Decision Pipeline, a live clinic-day simulation and dashboard, a command palette, voice input, a one-click auto-demo, an installable offline app and a Prompt Lab page. I also added the company's own evaluation criteria and submission requirements to the prompt.	Visual redesign and signature-feature round. The result is in the final repository.
6	30 Oct 2026, 16:17	AI coding tool (Codex or the second tool)	**Max-upgrade prompt.** Asked for as many relevant features and UI/UX improvements as possible: counterfactual explainer, safety-gated Rules Lab, FAQ-grounded replies, appointment booking, complaint and language handling, reply studio, spike alert, illustrative FHIR-style export and a 1,500-enquiry evaluation set. I added a rule that the video, my reflections and my prompt history stay my own work.	Feature round. The result is in the final repository.
7	30 Oct 2026, 16:31	AI coding tool (Codex or the second tool)	**Round 2 prompt (condensed, safe to re-run).** A shorter version I could feed again without redoing finished work: it first reads the project state, lists what is done, broken and missing, then builds only what is missing. It added a Trust Center page with in-browser self-tests.	Re-runnable upgrade round. The result is in the final repository.
8	30 Oct 2026, 17:29	AI coding tool (Codex or the second tool)	**Final optimisation prompt.** Last round before submitting: finish the feature set, polish the design and copy, test robustness, accessibility and security, run an independent audit from a fresh copy, and prepare my own submission guide.	Final polish and audit round. The result is in the final repository.

Prompts 5 to 8 were drafted by Claude at the times shown and used in this order as successive upgrade rounds. The coding tool used for each of them is not separately recorded.

3. How my prompts improved

- **Prompt 1 (build):** one prompt covering design, classifier, tools, tests and documents in phases. It set safety rules up front (synthetic data only, no medical advice, a human approves every reply, emergency wording always wins) and asked for a factual BUILD_LOG.md so real problems would be recorded instead of invented.
- **Prompt 2 (upgrade):** after reading the first result, I asked for an audit against the original plan first, a single rules file shared by JavaScript and Python with a parity test, a static mode that works on GitHub Pages, and measurable evaluation (emergency recall must stay at 100%).
- **Prompts 3 and 4 (token limit):** when the first tool ran low on tokens, I wrote prompts so a second tool could continue safely: inspect first, write a state file, avoid redoing finished work, fix what is broken before adding extras, and mark anything unsure as unverified.
- **Prompt 5 (design):** after seeing the app, I judged the design too plain and asked for a distinctive look and signature features. I also fed the company's own scoring criteria into the prompt so the work would map to what is assessed.
- **Prompts 6 and 7 (features and re-runs):** I added explicit ownership rules (my video, my reflections and my prompt history stay mine), priority orders for when tokens run short, and a "read the state first" step so the same prompt could be fed again without duplicating work.
- **Prompt 8 (final):** finish, polish, audit from a fresh copy, and prepare the submission.

Techniques that carried through every round: one source of truth for rules; safety constraints written as non-negotiable; "never invent numbers" and "do not claim anything passes without running it"; priority lists and stop conditions for limited tokens; and a clear split between what the AI does and what I do myself.

4. Messages I sent to Claude (planning and prompt drafting)

When (IST)	Message
30 Oct, 14:19	Pasted the company's Task 1 email and attached the Task 1 PDF (no separate question).
30 Oct, 14:22	"This is a new project built from scratch. How, what are you giving, like read walkthrough.md, study.md? What is that? Give me a new prompt. This is another company's project, right?"
30 Oct, 14:33	Pasted the output of the first Codex build and wrote: "give a promot to make the project as much as realistic as possible and advanced so the revejwers will be impressed" (led to prompt 2).
30 Oct, 14:39	"Codex has around 25% of his free tokens left. So what I want now is like ask it to give a prompt like what were you doing, and I will run this prompt with another AI tool of the same folder so the AI tool can continue from where it left off. That kind of prompt I want you to give me."
30 Oct, 14:40	"Like, I want Codex to give the prompt. That prompt must be given by you, which asks Codex the prompt." (led to prompt 3)
30 Oct, 14:47	Pasted an error from the coding tool: "Your access token could not be refreshed because you have since logged out or signed in to another account. Please sign in again.??" (led to prompt 4)
30 Oct, 15:14	Pasted the second tool's summary of what it finished and wrote: "See, the project and the website looks just like a normal website. That's it, nothing much of a out of the box kind of a thing. It just

	looks like a normal thing where I give input, it shows output with some colorful displays. That's it, not even colorful, it's just a bland design, whatever. Give a prompt saying that it should be out of the box. Do as much as you can and do as quickly as you can. Also, I will share the, I shared the PDF, right, which I got from the company. Add the also those things that these are the things that they met, and you need to do it as soon as possible because this is the first come, first serve opportunity, and I need to give the best results with the least amount of time. So give me the prompt." (led to prompt 5)
30 Oct, 15:18	"See, like I'm going to create a new repository for this project in my GitHub. So basically give me the title and the description. And after the project is complete, I will directly commit it. So now give me the name and the description."
30 Oct, 16:17	"Give me the next prompt saying upgrade as much as possible. I need more upgradation and I will submit everything by today evening or today night around 9. So what I want is the prompt condensed that upgrade as much as possible, add features, add relevant features related to the project, improve the UI UX if necessary, upgrade as much as possible. It should be in a way that the reviewers are impressed and they consider me for the next round. And also make sure of one thing, that the video message all will be done by myself. No need of you doing it. I will record myself saying what are the things done, everything. The things which are supposed to be done by me, I will do. And the things you are supposed to do is just upgrade, upgrade, upgrade. Add this thing to the prompt." (led to prompt 6)
30 Oct, 16:31	"agian optimixe this prompt so i can feed it again" (led to prompt 7)
30 Oct, 17:29	"You write another prompt saying this is the final optimization. Do as much as good updates or whatever possible, and then give after everything, give me the steps to submit and what are the things I need to fill, and give me the format and everything like a guide from submission to the till the end in a single PDF. But before doing the PDF, just upgrade everything as much as possible and make it a submittable project. Give me that prompt." (led to prompt 8)
31 Oct, 09:18	Asked Claude to create this prompt-documentation PDF and to list the steps I still need to do for the submission.

Messages are quoted as I typed them, including spelling mistakes. Long pasted tool outputs are summarised in one line rather than reproduced.

5. Full prompt texts

Each prompt below is shown exactly as it was given to the AI coding tool.

Prompt 1 Initial build prompt

Requested: 30 Oct 2026, 14:22 IST | Tool: Codex

```
# ONE-SHOT PROMPT FOR CODEX: "ClinicDesk Triage Assistant" (new project, new empty folder)

## HOW TO WORK
You are building a complete NEW project from scratch in the current EMPTY folder, autonomously, in one run.
Do not stop for approval and do not ask me questions. If something is ambiguous, choose a sensible
assumption, write it in docs/assumptions.md and continue.
- Run `git init` first (default branch `main`). Make a meaningful git commit at the end of each phase.
Never push anywhere.
- After each phase, run what you built and fix problems yourself. Never leave a phase half-working. Never
claim something passes without running it.
- Keep a running file `BUILD_LOG.md`: every real problem you hit while building (bug, failing test, bad
assumption, design change), what caused it, and how you fixed it. Be factual. This file is the evidence for
my video, so do not invent problems; only log real ones.
- Keep code simple and readable, with comments that explain the "why".
- The repo will be PUBLIC and reviewed by a hiring team. No secrets, no API keys, no real patient or
personal data, no phone numbers or emails of real people, no local file paths.
- At the end, print a final summary and a numbered "next steps for me" checklist (see the last section).

## PROJECT IN ONE PARAGRAPH
A small, working web tool that helps the front desk of a general-practice (GP) clinic in Australia handle
incoming patient enquiries. A message comes in; the tool sorts it by category and priority, spots emergency
wording and immediately tells the sender to call 000, explains which words triggered its decision, drafts a
polite reply for a staff member to approve, logs every step in an append-only history, and shows a small
dashboard. It uses only synthetic (fake) data. It never gives medical advice. A human staff member always
approves replies. Domain: healthcare. Area: automation + a working prototype, with AI used meaningfully to
build and (optionally) to assist at runtime.

## TECH CHOICES (keep it easy for reviewers to open and run)
- Front end: plain HTML, CSS and JavaScript (ES modules), NO build step, NO external CDN or framework. It
must work by opening `index.html` or from GitHub Pages. Put `index.html` at the repo root, plus a
`.nojekyll` file.
- Shared rules: all classification rules live in ONE file, `rules.json` (single source of truth).
- Classifier: `assets/classifier.js` (ES module that works in the browser and in Node).
- Python 3.10+ tools in `tools/`, standard library plus pytest only: synthetic data generator, batch triage
script, evaluator.
- Persistence in the browser: localStorage wrapped in try/catch (the app must still work if storage is
blocked).

## LAYOUT
```
README.md LICENSE(MIT) PROMPTS.md AI_USAGE.md BUILD_LOG.md VIDEO_SCRIPT.md SUBMISSION_EMAIL.md
index.html .nojekyll rules.json
assets/ classifier.js app.js storage.js styles.css samples.js(optional)
tools/   generate_enquiries.py triage_batch.py evaluate.py parity_check.js
data/   synthetic_enquiries.csv synthetic_enquiries_labels.csv   (labels are separate from inputs)
tests/   test_*.py
docs/   assumptions.md safety_notes.md evaluation_report.md architecture.md
prompt_history/   (I will add my prompt files here; create the folder with a README note)
```

---
## PHASE 1: Design and safety rules
1. Write docs/architecture.md (a Mermaid flow diagram: enquiry in, classify, red-flag check, draft reply,
human approval, log, dashboard) and docs/assumptions.md.
2. Write docs/safety_notes.md: this is a DEMO, not a clinical tool; it gives no diagnosis or medical
advice; emergency wording always wins over every other rule; a human approves every reply; the default mode
sends no data anywhere; do not type real patient information into the app.
```

```
3. Define the data model for an Enquiry (id like ENQ-000001, received_at, channel [web_form, email, sms,
phone_note], message, category, priority, matched_terms, confidence, status, draft_reply, approved_by,
approved_at) and an append-only EventLog (id like EVT-000001, enquiry_id, timestamp, event_type, details).
State changes are NEW log events, history is never overwritten.
4. Design rules.json: categories = appointment, prescription_refill, test_results, referral_letter,
billing_admin, general_info, urgent_clinical, other. Priorities = emergency, urgent, routine. Each category
has weighted phrases; emergency/urgent red-flag lists; negation words and a short window rule; reply
templates per category; service-level targets per priority (emergency: show 000 message instantly; urgent:
same day; routine: 2 business days; make these configurable).
5. Emergency red flags to cover (examples; extend sensibly): chest pain or pressure, severe difficulty
breathing, signs of stroke (face drooping, slurred speech, sudden weakness), severe bleeding, unconscious
or unresponsive, seizure, anaphylaxis or throat swelling, thoughts of suicide or self-harm, overdose,
severe allergic reaction, high fever in a baby. For emergency or self-harm cases the response text must
say: if this is an emergency call 000 now; for mental-health distress mention Lifeline 13 11 14; for non-
emergency health advice mention healthdirect 1800 022 222. Put these numbers in rules.json as config
constants and add a comment "verify before real use".
6. Commit.

## PHASE 2: Classifier (the core)
1. Implement `assets/classifier.js` exporting `classify(message, rules)` returning: category, priority,
matched_terms (the exact phrases that fired, with their category/red-flag source), confidence (0-1,
documented formula), reasons (plain-English list), and flags such as `multi_intent`, `too_short`,
`negation_applied`, `needs_human_review`.
2. Behaviors that MUST hold:
   - Emergency red flags override all other categories.
   - Simple negation is handled: "no chest pain", "not having trouble breathing", "denies chest pain" must
NOT trigger emergency, but must set `negation_applied` and `needs_human_review`.
   - Benign look-alikes: "chest cold", "heart-shaped", "breathing exercises booklet" must not trigger
emergency.
   - Common typos and simple variants are tolerated (a small documented fuzzy match, e.g. edit distance 1
for words of 6+ letters).
   - Multi-intent messages (e.g. refill + appointment) pick the strongest category, set `multi_intent`, and
list the others.
   - Empty or whitespace-only input returns a clear validation result, not a crash; very long input (e.g.
5,000+ characters) is handled quickly and safely; HTML/script text in the message is treated as plain text
and never executed.
   - Low confidence goes to `other` with `needs_human_review` true. Never silently guess.
3. Unit-test the classifier with Node's built-in `node --test` if Node is available (put tests in
`tests/classifier.test.mjs`), or via tools/parity_check.js plus Python tests.
4. Commit.

## PHASE 3: Python tools and synthetic data
1. `tools/generate_enquiries.py`: with a fixed seed (20261030) and a `--count` parameter (default 300),
generate synthetic enquiries from templates with variety (different wording, typos, politeness, length,
channels, timestamps over 4 weeks including after-hours). Include dedicated edge-case groups: negated
emergencies, benign look-alikes, multi-intent, empty/very short, very long, mixed case and punctuation,
non-English greeting words, and emergency messages. Names must be fake and generic (no real people). Write
`data/synthetic_enquiries.csv` (inputs only) and `data/synthetic_enquiries_labels.csv` (expected category,
expected priority, edge_case_type), kept SEPARATE; the classifier and batch code must never read the labels
file.
2. `tools/triage_batch.py`: read an enquiries CSV and write a triaged CSV and a JSON summary using the SAME
`rules.json`. Implement the rules in Python to mirror classifier.js exactly.
3. `tools/parity_check.js` plus a pytest that, when Node is available, runs both implementations over the
full synthetic file and requires identical category, priority and red-flag results for every row. If Node
is not available the test is skipped with a clear message. Fix any mismatch by fixing the code, not by
loosening the test.
4. Commit.

## PHASE 4: Web app
Build `index.html` + `assets/app.js` with these screens or sections (single page, responsive, keyboard-
accessible, labelled form fields, good colour contrast, a visible "DEMO - synthetic data only - not medical
advice" banner):
1. New enquiry form: channel, optional sender label (fake), message. Submit runs the classifier
immediately. A "Load 10 sample enquiries" button and an "Import synthetic CSV" button.
2. Result card: category, priority badge, confidence, the matched terms highlighted in the message text,
plain-English reasons, flags, and the service-level target. Emergency priority shows a prominent, full-
width "Call 000 now" panel BEFORE anything else.
3. Draft reply panel: reply generated from the category template (no medical advice, polite, Australian
English), editable by staff, with an Approve button (requires entering a staff name; stored as approved_by)
and a "mark as sent (simulated)" action. Approval records an event; nothing is actually sent anywhere.
4. Queue: table of enquiries sorted by priority then time, filters by category, priority and status,
```

```
search, and a status workflow new, triaged, reply_approved, sent, closed.
5. Audit log: view the append-only event history per enquiry.
6. Dashboard: counts by category, priority and status, plus an average "time to triage/approve" (simulated
clock acceptable), drawn with plain CSS/SVG (no chart library).
7. Export queue and log to CSV; "Reset demo data" with a confirmation.
8. Optional AI assist (default OFF): a toggle "AI-assisted reply polish". If enabled, the user pastes their
own API key (kept in memory only, never saved, never logged, never committed) and the app sends ONLY the
draft reply text (never the original message) to the chosen provider to rephrase it more warmly, then shows
the result for staff review with a clear label "AI-assisted - review before approving". Implement a small
provider interface so one provider works; if the request fails, fall back to the template with a visible
notice. Show a privacy warning before enabling. The app must be fully functional with this feature off.
9. Make sure nothing in the app sends data over the network in the default mode. Verify this by searching
the code for fetch/XMLHttpRequest and documenting it in docs/safety_notes.md.
10. Run the app in a real headless browser if available in the environment (or at minimum load index.html
and exercise the main flows with a script) and confirm no console errors. Fix issues. Commit.

## PHASE 5: Evaluation and tests
1. `tools/evaluate.py`: run the triage on the synthetic set, compare with the separate labels, and write
`docs/evaluation_report.md` and `evaluation_metrics.json` with: overall category accuracy, per-category
precision and recall, priority accuracy, EMERGENCY RECALL (must be 100% on the emergency group; if it is
not, fix the rules), emergency false-positive rate, performance on each edge-case group (negation, look-
alike, multi-intent, empty, very long), the share routed to human review, and a confusion matrix. List
honest limitations (synthetic data, keyword rules are not a clinical model, English only, not validated by
clinicians).
2. pytest suite in `tests/`: ID formats; no crash on edge inputs (empty, huge, HTML/script text, unicode);
emergency always overrides; negation behavior; determinism of the generator (same seed gives identical
output, compare SHA-256 across two runs); labels file never imported by classifier or batch code (automated
source check); reply templates never contain medical-advice phrases from a documented blocklist; rules.json
validity (every category has templates).
3. Run everything: `pytest -q`, the Node tests/parity check, evaluate, and generate the data twice to prove
reproducibility. Commit.

## PHASE 6: Self-audit and fix (important: shows iterative improvement)
1. Act as an independent reviewer. Audit your own work against this checklist by running real commands:
fresh clone into a temp folder works; every README command works; app has no console errors and no network
calls by default; emergency recall 100%; parity check passes; tests pass; no secrets or personal data; no
file over 5 MB; docs match the code; accessibility basics (form labels, focus order, contrast); the app
works with localStorage blocked.
2. Fix every failure you find, re-run, and record each defect and fix in BUILD_LOG.md under a heading
"Self-audit round 1" (only real findings).
3. Commit.

## PHASE 7: Documentation and submission materials
1. README.md in plain words: what it is, who it helps, screenshots are optional (if you can capture them
headlessly, save in docs/screenshots/), how to run it (open index.html, or `python -m http.server`), how to
publish on GitHub Pages (Settings, Pages, deploy from branch `main`, root folder), how to run the Python
tools and tests, results with real numbers from evaluation_metrics.json, assumptions, limitations, safety
notice, next steps.
2. PROMPTS.md: the assessment requires ALL prompts I used, including significant revisions and follow-ups.
Create it with: an intro; a table with columns (No., date/time IST, tool, purpose, link, result); the first
row for THIS build prompt linked to `prompt_history/01_codex_build_prompt.md`; an empty "Revisions and
follow-up prompts" section with a ready-to-fill template (what was wrong, the prompt I sent, what changed);
and a section "Prompts sent to Claude (planning)" with a template. Leave all rows that depend on my own
history marked `TODO (owner)`. Do NOT invent prompts or dates.
3. AI_USAGE.md: tools used (Claude for planning and prompt drafting; Codex for building), where AI helped,
where I relied heavily on it (nearly all code), what I should personally verify, and a "TODO (owner)"
reflection section with 4 open questions for me to answer in my own words. Do NOT write my reflections for
me.
4. VIDEO_SCRIPT.md: a 3-minute video outline for me (timestamps) covering exactly what the assessment asks:
(1) AI tools used and why, (2) prompt engineering: how prompts were designed, refined and improved, (3) my
approach and how prior knowledge influenced decisions, (4) challenges and how they were solved (pull these
ONLY from the real entries in BUILD_LOG.md), (5) a live demo walkthrough with the exact click-by-click
steps and sample messages to paste (include one emergency, one negated emergency, one multi-intent, one
refill, one billing). Leave sentences about my personal background as `[SAY IN YOUR OWN WORDS]`
placeholders. Include recording tips (screen recorder, mic check, keep under 5 minutes).
5. SUBMISSION_EMAIL.md: a draft email to jobs@gokulinfocare.com with a clear subject (e.g. "Task 1
submission - AI Growth & Automation Associate - Srajan Shetty"), a short body summarizing the project in 3
sentences, and three clearly labelled items: (1) video link [TODO], (2) prompt documentation link
(PROMPTS.md in the repo, plus the prompt_history folder) , (3) project link (GitHub repo and live GitHub
Pages URL) [TODO]. Remind to set any Google Drive link to "anyone with the link can view" and test it in an
incognito window.
```

```
6. Final consistency pass: grep the repo for TODO (owner) and list them; check README links; scan for
secrets, emails, phone numbers (apart from the public helpline numbers in rules.json), and local paths;
commit.

## DEFINITION OF DONE (verify each with a command)
1. Opening index.html works with zero setup; no external network calls in default mode.
2. Emergency recall is 100% on the synthetic emergency group, and negated-emergency and look-alike cases
behave as specified.
3. `pytest -q` passes; the Node/Python parity check passes; the generator is reproducible (identical SHA-
256 on two runs).
4. README, PROMPTS.md, AI_USAGE.md, BUILD_LOG.md, VIDEO_SCRIPT.md, SUBMISSION_EMAIL.md exist; only owner-
specific parts are left as TODO (owner).
5. Git has one meaningful commit per phase, the working tree is clean, and no large files or secrets are
tracked.

## FINAL MESSAGE TO ME (print this at the very end)
A numbered checklist of what I must do next, with exact commands, covering:
1. Put this prompt into `prompt_history/01_codex_build_prompt.md` and any later prompts I send to any AI
tool into PROMPTS.md (all prompts, plus revisions).
2. Run the app and tool myself, try the 5 demo messages, and read README and docs/architecture.md so I can
explain it.
3. Answer the reflection questions in AI_USAGE.md in my own words.
4. Create an empty PUBLIC GitHub repo and push (`git branch -M main`, `git remote add origin ...`, `git
push -u origin main`), enable GitHub Pages (Settings, Pages, branch main, root), and open both the repo and
the live URL in an incognito window.
5. Record the 2+ minute video following VIDEO_SCRIPT.md, upload it (for example to Google Drive or YouTube
unlisted), set access to anyone with the link, and test the link in incognito.
6. Fill the links into SUBMISSION_EMAIL.md and email jobs@gokulinfocare.com. It is first-come first-served,
so send as soon as everything is checked.
Also print a verdict: READY AFTER MY TODOs, or the exact blockers.

Begin now with Phase 1 and run through Phase 7 without stopping.
```

Prompt 2 Upgrade prompt (realistic and advanced)

Requested: 30 Oct 2026, 14:33 IST | Tool: Codex

```
# UPGRADE PROMPT FOR CODEX: make the Clinic Enquiry Desk realistic, advanced and review-ready

## HOW TO WORK
Continue in the current project folder (the Clinic Enquiry Desk prototype you already built). Work
autonomously and do not ask me questions. Keep everything that already works: do not break existing
features, and re-test after every phase.
- Never claim something works without running it. Every number in docs, README or reports must come from a
real run. No invented metrics, no claims of clinical validation or regulatory approval.
- Keep a factual `BUILD_LOG.md` (append a new section "Upgrade round"): every real problem hit, its cause
and its fix.
- Git: first run `git status`. If git commits work, make one meaningful commit per phase. If `.git` is
read-only or commits fail, do NOT keep retrying: note it once in BUILD_LOG.md, continue the work without
commits, and tell me at the end. Never push.
- The repo is PUBLIC and will be reviewed by a hiring team. No secrets or API keys, no real patient data,
no real people's names/phones/emails, no local file paths. All data is synthetic. This is a demo, not a
medical device, and it never gives diagnosis or medical advice. Emergency wording always wins over every
other rule and a human approves every reply.
- Zero-install must still work: `npm start` runs the app using only Node's standard library. Python tools
use only the standard library plus pytest.

## PHASE 0: Audit what exists against the original plan, then close gaps
1. List the repo files and compare with this target: a browser app with a shared rules file (`rules.json`),
an explainable classifier, Python tools (`tools/generate_enquiries.py`, `tools/triage_batch.py`,
`tools/evaluate.py`), synthetic data with labels kept in a SEPARATE file, tests, docs (`architecture.md`,
`assumptions.md`, `safety_notes.md`, `evaluation_report.md`), and submission files (`README.md`,
`PROMPTS.md`, `AI_USAGE.md`, `BUILD_LOG.md`, `VIDEO_SCRIPT.md`, `SUBMISSION_EMAIL.md`, `LICENSE` MIT,
`prompt_history/`). Write the gaps to BUILD_LOG.md and create whatever is missing.
2. Move all triage rules into ONE `rules.json` read by both the JS classifier and the Python tools (single
source of truth), and add a parity test: the JS and Python implementations must give identical category,
priority and red-flag results for every row of the synthetic dataset.
3. Run the app, fix anything broken, run a syntax check on every JS file. Commit.

## PHASE 1: Dual mode so reviewers can open it with one click
1. Keep the Node server as "Server mode" (persistent hash-chained event log in a local file).
2. Add "Static demo mode": the same UI and classifier run fully in the browser with localStorage (wrapped
in try/catch so the app still works if storage is blocked), deployable to GitHub Pages with `index.html` at
the repo root and a `.nojekyll` file. The UI detects which mode it is in and shows a small badge ("Demo
mode: runs in your browser, nothing is sent anywhere").
3. Document both in README (open index.html or `python -m http.server`; `npm start`; how to enable GitHub
Pages: Settings, Pages, branch main, root).
4. Prove with a search of the code that in default mode the app makes no network calls, and document it in
docs/safety_notes.md. Commit.

## PHASE 2: Realistic clinic workflow (the "feels like a real tool" layer)
Implement these, each configurable through a `config/clinic.json` (fictional clinic name, opening hours per
weekday, public holiday dates, service-level targets, staff roster with roles):
1. Roles and routing: Receptionist, Practice Nurse, GP, Practice Manager. Routing rules send categories to
the right queue (refill requests to GP review, test results to Nurse, billing to Reception, urgent clinical
to Nurse with GP escalation). A role switcher in the UI (demo only, clearly labelled as simulated login)
changes the visible queue and which actions are allowed (for example only a GP can approve a prescription-
refill reply; reception can approve admin replies). The server mode must enforce the permission rules on
the API, not only in the UI, with tests.
2. Business hours: classify arrival as in-hours or after-hours using the clinic config; after-hours routine
messages get an after-hours auto-acknowledgement template; emergency wording always shows the 000 panel
regardless of hours.
3. Service-level timers: per-priority targets, a live countdown or "overdue" badge, breach highlighting,
and a pinned "Unacknowledged emergency flags" banner that stays until a staff member acknowledges it
(acknowledgement is recorded as an event).
4. Identity-safe handling: detect and mask sensitive patterns in stored/displayed log text (Australian-
style phone numbers, Medicare-like number patterns, dates of birth, email addresses) and show a gentle
warning that sensitive details should not be sent by email/SMS. Show a "verify patient identity (name +
DOB)" checklist before any reply that mentions results or prescriptions can be approved. All data is
fictional.
5. Duplicate and repeat detection: flag likely duplicates or repeated enquiries from the same fictional
```

```
sender within a window and link them in the UI.
6. Reply quality: reply templates in plain Australian English with a readability score check (simple
documented formula), a length limit for SMS-channel replies, and a blocklist test proving templates never
contain medical-advice wording.
7. Audit and integrity: keep the append-only, hash-linked event log (server and demo mode), and add a
"Verify log integrity" button plus a clearly labelled "Tamper demo" in a temp copy that shows the
verification failing when one event is edited. Include the verification logic in tests.
8. Reporting: a manager view with volume by hour of day, backlog ageing, SLA compliance %, median time to
first response, workload by staff member, plus CSV export of queue and log. Commit.

## PHASE 3: Optional hybrid AI layer, built safely
1. Add a provider interface `assets/ai/` (or `server/ai/`) with three providers: `none` (default), `mock`
(deterministic, offline, used in tests and demos), and one real provider that reads the key from an
environment variable in server mode or from a user-entered, memory-only field in demo mode. Keys are never
saved, logged or committed. Show a privacy warning before enabling any real provider, and send only the
minimum necessary text.
2. "Second opinion" mode: the AI returns STRICT JSON (category, priority, reasons). Validate it against a
schema; invalid output is discarded. Merge rule (safety first): the AI may only UPGRADE priority or add a
review flag, never downgrade a rule-based emergency or urgent result. Any disagreement between rules and AI
sets `needs_human_review` and shows both opinions side by side with an explanation.
3. Prompt-injection hardening: the patient message is treated as data, never as instructions. Add a test
set of injection attempts (for example text telling the model to ignore its instructions, to mark
everything routine, or to reveal its prompt) and prove with the mock provider and the validation layer that
they cannot lower priority or break the schema.
4. Everything must work with the AI layer off. Document the design in docs/architecture.md. Commit.

## PHASE 4: Evaluation that stands up to scrutiny
1. Regenerate the synthetic dataset with a fixed seed (20261030) at 1,000 enquiries with much more variety:
different tones, typos, abbreviations, ESL phrasing, long messages, multi-intent, after-hours timestamps,
negation ("no chest pain"), benign look-alikes ("chest cold"), borderline urgency, and
adversarial/injection-style text. Labels live in a separate file that triage code never reads (add an
automated source check).
2. `tools/evaluate.py` reports, from real runs: category accuracy and per-category precision/recall,
priority accuracy, EMERGENCY RECALL (must be 100% on the emergency group; if not, fix the rules), emergency
false-positive rate, results per edge-case group, human-review rate, confusion matrix, confidence
calibration, and bootstrap 95% confidence intervals for the headline metrics. Compare rules-only vs
rules+mock-AI second opinion.
3. Add an error analysis section to docs/evaluation_report.md: the 10 worst misclassifications with
explanations, what was changed to fix them, and honest limitations (synthetic data, keyword rules, English-
focused, not clinically validated). Include a latency benchmark (classification time per message).
4. Reproducibility proof: generate twice with the same seed and compare SHA-256 of the outputs, and show a
different seed produces different data. Commit.

## PHASE 5: Engineering quality and safety
1. Tests: unit tests for the classifier and routing, API integration tests for permissions and approvals,
parity test, integrity-chain test, privacy-masking test, injection tests, hours/SLA tests, determinism
test. Use `node --test` and pytest. Report the real pass counts.
2. Security basics for server mode: strict input validation and size limits, output escaping (no XSS, test
with script text), security headers, simple rate limiting, safe JSON error responses with no stack traces,
path-traversal protection for static files, no eval.
3. Accessibility pass: labelled form controls, keyboard operation of every action, visible focus, colour
contrast, `aria-live` for the emergency banner and status changes, reduced-motion respect. Light and dark
theme.
4. Add a GitHub Actions workflow `.github/workflows/ci.yml` that runs the tests on push (Node and Python).
Add a Dockerfile for server mode (documented as optional, no need to build it here if Docker is
unavailable) and a `.gitignore`.
5. If a headless browser is available, run an end-to-end script through the main flows (submit an
emergency, approve a reply, switch role, verify log integrity) and confirm there are no console errors; if
not available, say so in BUILD_LOG.md. Commit.

## PHASE 6: Demo polish (what makes reviewers remember it)
1. "Simulate a clinic day" button: generates a realistic stream of fictional enquiries over a simulated day
(morning peak, lunchtime lull, after-hours trickle, an occasional emergency), with a speed control, and
shows the queue, SLA timers and dashboard reacting live. Deterministic with a seed.
2. A guided tour (short step-by-step overlay or a "Demo script" panel) walking through: emergency handling,
negated emergency, multi-intent, refill approval requiring the GP role, identity checklist, integrity
verification.
3. Capture 4-6 screenshots into docs/screenshots/ if a headless browser is available and reference them in
the README. Never fabricate screenshots.
4. A visible "Why this decision?" panel on every result showing matched phrases, rules fired, confidence
formula and routing reason. Commit.
```

```
## PHASE 7: Documentation, safety case and submission materials
1. docs/architecture.md: updated Mermaid diagrams (data flow, roles and permissions, hybrid AI merge rule, event log integrity).
2. docs/safety_notes.md plus a simple hazard log table (hazard, example, mitigation implemented, residual risk) covering: missed emergency, false reassurance, wrong routing, privacy leakage, AI hallucination or injection, stale SLA timers, staff over-trust. State plainly that this is a demo and list questions a real deployment would need to answer (clinical safety review, privacy law, medical-device regulation) WITHOUT making legal claims.
3. docs/evaluation_report.md and `evaluation_metrics.json` with the real numbers.
4. Update README: what it is, who it helps, demo mode vs server mode, how to run everything, results table from the real metrics, limitations, next steps.
5. PROMPTS.md: add this upgrade prompt as a revision entry (what the first build lacked and why this follow-up was written), linking `prompt_history/02_upgrade_prompt.md` if that file exists. Leave my own history marked `TODO (owner)`. Do NOT invent prompts or dates.
6. AI_USAGE.md: tools used, where AI accelerated work, where I relied heavily on it, what I need to verify personally; keep the reflection questions as `TODO (owner)`.
7. VIDEO_SCRIPT.md: update the 3-minute outline to include the new features in a tight demo order, and fill the challenges section ONLY from real entries in BUILD_LOG.md. Keep my personal sentences as `[SAY IN YOUR OWN WORDS]`.
8. SUBMISSION_EMAIL.md: keep the draft to jobs@gokulinfocare.com with placeholders for the video link, the prompt documentation and the project links. Commit.

## PHASE 8: Independent self-audit and final report
1. Audit the repo like a skeptical reviewer, with real commands: fresh copy in a temp folder runs; every README command works; tests pass; emergency recall is 100%; parity passes; no network calls in default mode; no secrets, emails, personal phone numbers or local paths; no file over 5 MB; docs match the code; app works with localStorage blocked; permissions enforced by the API.
2. Fix every failure, re-run, and log each defect and fix in BUILD_LOG.md under "Self-audit".
3. Write `docs/FINAL_AUDIT.md` with a table of every check, PASS or FAIL, and evidence.

## SCOPE RULES
- Priorities if time runs short: Phase 0, 1, 2 (items 1-4 and 7), 4, 5 first; then the rest. Never leave a phase half-finished; skip a feature rather than ship it broken, and list skipped items in README under "Not done yet".
- Do not add heavy dependencies. Do not add real medical content beyond the red-flag phrase lists and helpline numbers already in rules.json.

## FINAL MESSAGE TO ME
Print: what was built, the real metrics, what was skipped, the git status (and whether commits succeeded), and a numbered checklist of my next steps with exact commands: run and try the app in both modes; read README and docs/architecture.md so I can explain it; save this prompt as prompt_history/02_upgrade_prompt.md and log every prompt I used in PROMPTS.md; answer the reflection questions in AI_USAGE.md myself; commit and push to an empty PUBLIC GitHub repo; enable GitHub Pages; check repo and live URL in incognito; record the 2+ minute video following VIDEO_SCRIPT.md; email jobs@gokulinfocare.com. End with a verdict: READY AFTER MY TODOs, or the exact blockers.

Begin now with Phase 0 and run through Phase 8 without stopping.
```

Prompt 3 Handoff request (token limit)

Requested: 30 Oct 2026, 14:40 IST | Tool: Codex

```
I'm almost out of tokens, so I'm switching to another AI tool in this same folder. Write a handoff prompt
for it so it can continue exactly where you stopped.

Be economical: do NOT run long test suites or rebuild anything. Only use cheap commands: git status, a file
listing, `node --check` on the JS files, and reading the tail of BUILD_LOG.md. First make sure every file
you were editing is saved and nothing is half-written; if something is half-written, finish that one piece
or clearly mark it.

Create HANDOFF_PROMPT.md in the project root. It must be a complete, self-contained prompt addressed to the
next AI tool, covering:
1. Goal and context: the Clinic Enquiry Desk project (synthetic data only, no medical advice, human
approves every reply, emergency wording always wins), that this is a hiring assessment for a public GitHub
repo, and what the reviewers need: working project link, prompt documentation, a 2+ minute video by me.
2. Current state, split into three clearly labelled lists: DONE AND VERIFIED (say how you verified it),
DONE BUT NOT VERIFIED, and IN PROGRESS / NOT STARTED. Compare against the phases of the upgrade prompt
(Phase 0 to 8) and say which phase you were in and what step.
3. Exact file layout, what each important file does, and the commands to run the app and any tests.
4. Known bugs, failures, skipped items and decisions you made (and why).
5. Git state: whether commits work, what is uncommitted.
6. Rules for the next tool: read BUILD_LOG.md, docs/assumptions.md and docs/safety_notes.md first; do not
redo finished work; do not break existing features; verify before claiming anything passes; log every real
problem in BUILD_LOG.md; never invent metrics; no secrets, real patient data or local paths; leave my owner
TODOs (reflections, my own prompt history, video words) untouched.
7. Remaining work as an ordered, numbered task list, most important first, with a note on what to cut if
the next tool also runs low on tokens. Submission files still to finish (PROMPTS.md, AI_USAGE.md,
VIDEO_SCRIPT.md, SUBMISSION_EMAIL.md, README) must be included.
8. A final instruction: when finished, print my numbered next-steps checklist (run both modes, answer
reflections, push to public GitHub, enable GitHub Pages, incognito check, record the video, email
jobs@gokulinfocare.com).

Keep it factual. Mark anything you are unsure about as UNVERIFIED instead of guessing. Do not include
secrets. When done, print only the path of the file and a 5-line summary of where you stopped.
```

Prompt 4 Continue-without-notes prompt

Requested: 30 Oct 2026, 14:47 IST | Tool: Second AI coding tool (its output footer shows the model name “Big Pickle”)

```
This folder contains a half-finished hiring-assessment project: "Clinic Enquiry Desk", a clinic enquiry
triage prototype. It uses synthetic data only, gives no medical advice, a human approves every reply, and
emergency wording always wins. A previous AI tool (Codex) built it and ran out of tokens, so you are
continuing its work. I have no handoff notes, so work out the state yourself with cheap checks only.

Step 1, inspect: list the files, run git status, run `node --check` on every JS file, and read
BUILD_LOG.md, docs/assumptions.md and docs/safety_notes.md if they exist. Then write HANDOFF_STATE.md with
three lists: DONE AND VERIFIED (say how you verified), DONE BUT UNVERIFIED, and MISSING. Mark anything you
are unsure about as UNVERIFIED.

Step 2, compare with the target: a browser app with one shared rules.json, an explainable classifier,
Python tools (generate_enquiries.py, triage_batch.py, evaluate.py), synthetic data with labels in a
SEPARATE file, tests, docs (architecture, assumptions, safety_notes, evaluation_report), static demo mode
that works on GitHub Pages (index.html at repo root, .nojekyll) plus the Node server mode, and the
submission files: README.md, PROMPTS.md, AI_USAGE.md, BUILD_LOG.md, VIDEO_SCRIPT.md, SUBMISSION_EMAIL.md,
LICENSE (MIT).

Step 3, finish in this priority order and stop when tokens run low: (1) anything broken in the app; (2)
static demo mode; (3) rules.json shared by JS and Python, synthetic dataset with a separate labels file,
and an evaluation with emergency recall at 100%; (4) tests; (5) the submission files; (6) extras like roles
and routing, SLA timers, sensitive-data masking, a verify-log-integrity button, and a simulate-a-clinic-day
button.

Rules: don't redo finished work or break what works; verify before claiming anything passes; log every real
problem in BUILD_LOG.md; never invent metrics; no secrets, real patient data or local paths; leave my owner
TODOs (my reflections, my own prompt history, my video wording) untouched. If git commits fail, tell me
once and continue. Finish with a numbered checklist of my next steps: run both modes, answer the
reflections, push to a public GitHub repo, enable GitHub Pages, check in incognito, record a 2+ minute
video, and email jobs@gokulinfocare.com.
```

Prompt 5 Wow-factor design prompt

Requested: 30 Oct 2026, 15:14 IST | Tool: AI coding tool (Codex or the second tool)

```
# "WOW FACTOR" UPGRADE PROMPT: make the Clinic Enquiry Desk memorable, fast

## CONTEXT (read carefully)
This is a hiring assessment (AI Growth & Automation Associate, a healthcare-technology company in
Australia). The submission is first-come, first-served, so SPEED matters: finish a polished, working result
in about 45-60 minutes of work. Work autonomously and do NOT ask me questions. Do not redo anything that
already works.

The company will judge:
- Practical use of AI tools
- Prompt engineering skills and iterative improvement
- Creativity and innovation
- Problem-solving and technical execution
- Quality and usefulness of the final outcome
- Ability to explain decisions and demonstrate what was built
They said: "Don't just tell us what AI can do. Show us what you can create with it."
They require three things: (1) a video of at least 2 minutes (AI tools used and why; prompt design and
refinement; approach and prior knowledge; challenges and how they were solved; final demo), (2) prompt
documentation with all prompts and significant revisions, (3) a project link (public GitHub repo and,
ideally, a live website). Reviewers will click the live link first, so the first 10 seconds must impress
and everything must work with no setup.

The current app works but looks like a bland generic form-and-results page. Your job: turn it into
something that feels like a real, distinctive product and that clearly shows creativity and good
engineering, without breaking anything.

## NON-NEGOTIABLE RULES
- Keep existing features and tests working. Run `node --check` on every JS file, `npm test`, and the Python
tests after the big steps, and fix failures.
- Be token-economical: make focused edits, do not rewrite whole files unnecessarily, do not run long
outputs. If you are running low on tokens, STOP at a stable, working state and make sure README and
VIDEO_SCRIPT.md are updated.
- No external CDNs, no external fonts, no frameworks, no build step. Plain HTML, CSS, JS with inline SVG.
Must work on GitHub Pages and from file://.
- Safety rules stay: synthetic data only, no medical advice, a human approves every reply, emergency
wording always wins, no network calls in default mode, no secrets, no real personal data, no local paths.
Respect `prefers-reduced-motion` and keep keyboard accessibility and colour contrast.
- Every number shown in the UI or docs must come from real data or be clearly labelled as an adjustable
assumption. No fake metrics, no fake testimonials, no fake screenshots.
- Log every real problem and fix in BUILD_LOG.md under "Wow round". If git commits work, commit after each
step; if they fail, note it once and continue.

## STEP 1: A distinctive design system (about 10 minutes)
Replace the bland look with a coherent "clinic mission control" visual identity:
1. CSS variables for a dark navy base with a teal accent, amber for warnings and a strong red for
emergencies; also a clean light theme with a toggle (remember the choice with try/catch). Large, confident
typography using the system font stack, tabular numerals for counters, generous spacing, rounded cards with
soft depth, subtle gradients, status colours that never rely on colour alone (icons and text labels too).
2. A hero header with the product name, a one-line value statement, a live status strip (mode badge, clock,
in-hours/after-hours state, queue size) and a prominent "Watch the 60-second auto-demo" button.
3. Inline SVG icons (inbox, shield, route, clock, user, check, alert) written by hand. Smooth micro-
interactions (hover lift, animated badges, number count-up) that switch off for reduced motion.
4. Responsive layout that looks good on a phone and on a wide screen. Commit.

## STEP 2: The "Decision Pipeline" animation (the signature feature, about 10 minutes)
When an enquiry is submitted, show an animated horizontal pipeline that lights up stage by stage with real
data from the classifier, so reviewers literally SEE how the decision is made:
Intake, then Privacy mask (for example "1 phone number masked"), then Understand (matched terms highlighted
inside the message, category with confidence ring), then Safety check (red-flag phrases, negation handling
shown explicitly, for example "'no chest pain' detected, negation applied, sent to human review"), then
Route (which role queue and why), then SLA clock (target time with a countdown), then Draft reply, then
Human approval gate.
Each stage shows a one-line plain-English explanation and the exact rule or phrase that fired. Add a
"Replay" button and a step-through mode. For emergencies, the safety stage turns into a full-width pulsing
```

```
red "CALL 000 NOW" panel with the helpline lines, a large-type read-aloud button (browser speech synthesis, triggered only by the user), and a required staff acknowledgement. Commit.

## STEP 3: Mission-control dashboard and live simulation (about 10 minutes)
1. Make "Simulate a clinic day" much more visual: a speed control, a live arrival timeline across the day (morning peak, lunch lull, after-hours), a queue split into swimlanes by role, animated cards that move as statuses change, a live SLA-breach counter, and a pinned emergency alert strip.
2. Charts drawn with plain SVG/CSS only: volume by hour heatmap, priority mix donut, backlog ageing bars, SLA compliance gauge, median time to first response. Everything computed from the real event data in the app.
3. A "Manual vs triage-assisted" comparison panel with an adjustable assumption slider (minutes of staff time per enquiry, clearly labelled "assumption, not measured") that shows estimated staff time saved, never presented as a measured result. Commit.

## STEP 4: Out-of-the-box interactions (about 10 minutes)
1. Command palette (Ctrl+K or "/"): jump to a queue, switch role, load a sample, run the auto-demo, toggle theme, verify log integrity.
2. Voice input for "phone notes": a microphone button using the browser speech recognition API where available (feature-detected; otherwise hide it and explain), so a receptionist can dictate a message. Processing stays in the browser.
3. One-click "Auto-demo" (60 seconds, skippable): a scripted tour that submits a clear emergency, a negated emergency ("no chest pain"), a benign look-alike ("chest cold"), a multi-intent message, a refill that needs the GP role and the identity checklist, then verifies log integrity and shows the dashboard, with captions explaining each step. Reviewers who do not interact must still understand everything.
4. "Verify integrity" and the tamper demo shown visually (a chain of blocks that turns red at the broken link). Commit.

## STEP 5: Product polish that reads as "real" (about 10 minutes)
1. Installable offline app: add `manifest.webmanifest`, icons made with SVG/inline data, and a simple service worker that caches the static files (safe for GitHub Pages subpaths; wrap registration in try/catch and make sure the app works without it).
2. One-click "Shift handover report": a printable page (print stylesheet) summarising the day: counts, emergencies and how they were handled, overdue items, items awaiting approval, all from the real event log.
3. A "Prompt Lab" page inside the app that shows how this project was built with AI: it lists the real prompt files found in `prompt_history/` (title, purpose, and the real outcome recorded in BUILD_LOG.md or the audit files, such as defects found and fixed) and the evolution of the triage rules and safety decisions. Use only real content; if a source file is missing, leave a clear "TODO (owner)" note instead of inventing it.
4. A "Safety case" page summarising the hazard log and the safety-first merge rules in plain English, and a "Limits" page stating plainly that this is a demo and not a clinical tool. Commit.

## STEP 6: Map everything to the company's criteria and finish the submission files (about 10 minutes)
1. README.md: add a short "Try it in 60 seconds" section at the top (live link placeholder, auto-demo button, 3 sample messages) and a table titled "How this maps to the assessment criteria": each criterion above, where in the project it is shown, and the real evidence (test counts, evaluation metrics, BUILD_LOG entries). Keep claims honest and add a "Not done yet" list.
2. VIDEO_SCRIPT.md: rewrite as a tight 3-minute script that covers, in order, the five topics they require: AI tools and why; prompt engineering (initial prompt, audit round, upgrade rounds, what each improved, using real examples from prompt_history and BUILD_LOG.md); approach and how prior knowledge influenced decisions (leave `[SAY IN YOUR OWN WORDS]` placeholders); challenges and how they were solved (only real ones from BUILD_LOG.md); final demo using the auto-demo and the Decision Pipeline. Add click-by-click timings and recording tips.
3. PROMPTS.md: add this prompt as a new revision entry (what was wrong: the first version looked generic, and why this follow-up was written) and keep my own history marked `TODO (owner)`. Do NOT invent prompts or dates. Update AI_USAGE.md with an honest line about this round; leave my reflection questions as `TODO (owner)`.
4. SUBMISSION_EMAIL.md: keep the draft to jobs@gokulinfocare.com, add the live-site and repo placeholders.
5. If a headless browser is available, take 4-6 real screenshots (hero, Decision Pipeline during an emergency, dashboard during a simulation, Prompt Lab) into docs/screenshots/ and reference them in the README; if not, say so in BUILD_LOG.md. Commit.

## FINAL CHECKS (run, do not assume)
`node --check` on all JS, `npm test`, Python tests, evaluation still at 100% emergency recall, static mode works from the repo root, from a subpath and from file://, zero `/api/` requests in static mode, no console errors in the main flows, no secrets or local paths. Fix failures.

## FINAL MESSAGE TO ME
Print: what changed, the real test and evaluation numbers, anything skipped, git status, and a short numbered checklist for me: run it in both modes and try the auto-demo; save this prompt as `prompt_history/03_wow_prompt.md` and log every prompt I used in PROMPTS.md; answer the AI_USAGE reflections in my own words; push to an empty PUBLIC GitHub repo; enable GitHub Pages (Settings, Pages,
```

```
branch main, root); open the live URL in incognito and click through the auto-demo; record the 2+ minute
video; email jobs@gokulinfocare.com with the repo link, live link, video link and the prompt documentation.
End with a verdict: READY AFTER MY TODOs, or the exact blockers.

Begin now with Step 1 and continue through Step 6 without stopping.
```

Prompt 6 Max-upgrade prompt

Requested: 30 Oct 2026, 16:17 IST | Tool: AI coding tool (Codex or the second tool)

```
# MAX UPGRADE PROMPT: Clinic Enquiry Desk (condensed)

## MISSION
Upgrade, upgrade, upgrade. Make this the most impressive, polished and genuinely useful project in a hiring assessment for an AI-skills role at an Australian healthcare-technology company. Reviewers score: practical AI-tool use, prompt-engineering iteration, creativity and innovation, technical execution, quality and usefulness, and explainability of decisions. Add relevant features, improve UI/UX everywhere, and strengthen quality. Time budget: about 2 hours of work in total. Work autonomously, ask no questions, and do not redo what already works.

## WHAT YOU DO AND DO NOT DO
- YOU: all building, upgrading, testing, documenting features.
- NOT YOU: the video. I record the video myself in my own words. Do not write, rewrite or add any video script or narration, and do not touch `VIDEO_SCRIPT.md` if it exists. Do not write my reflections or answers in `AI_USAGE.md`, and do not invent or edit my prompt history in `PROMPTS.md` or `prompt_history/` (you may add this prompt as one new revision row, factually, nothing else). Leave every `TODO (owner)` untouched.
- Instead, create `WHATS_NEW.md`: a factual reference for me listing every feature and improvement, where to find it in the UI, and exact click paths. It is a cheat sheet for my own recording, not a script.

## HARD RULES
- Keep everything that works; run `node --check` on all JS, `npm test`, and the Python tests after each tier and fix failures. Static mode must keep working on GitHub Pages (repo root), from a subpath and from file://, and make no network calls in default mode.
- Plain HTML/CSS/JS, inline SVG, no CDN, no external fonts, no frameworks, no build step. Python: standard library + pytest.
- Safety stays: synthetic data only, no medical advice, human approves every reply, emergency wording always wins and can never be downgraded by any feature, no secrets, no real personal data, no local paths.
- Every number in the UI/docs is real or a clearly labelled adjustable assumption. No fake metrics, fake testimonials or fake screenshots. Helpline numbers live in config with a "verify before real use" comment.
- Respect `prefers-reduced-motion`, keyboard access, visible focus and contrast. Log every real problem and fix in BUILD_LOG.md under "Max upgrade". Commit after each tier (if git fails, note once and continue). If tokens run low, stop at a stable state and write `HANDOFF_PROMPT.md` for the next tool.

## TIER 1: UI/UX overhaul (do first)
1. Distinctive design system: dark "mission control" theme + clean light theme with toggle; tabular numerals, strong hierarchy, cards with depth, icon set in inline SVG, status colours that also use icons/text, consistent spacing and components.
2. Navigation: left sidebar or top tabs (Inbox, Triage, Queue, Dashboard, Simulator, Rules Lab, Safety, Prompt Lab, Settings), command palette (Ctrl+K), keyboard-shortcut overlay (?), toasts, undo for status changes, skeleton/loading and helpful empty states, error states, onboarding hint cards on first visit (dismissible, remembered with try/catch).
3. Responsive for phone and widescreen; sticky emergency alert strip; polished forms with inline validation.
4. Hero/landing section with a one-line value statement, live status strip and a 60-second auto-demo button (scripted scenario with captions: emergency, negated emergency, look-alike, multi-intent, refill needing GP role plus identity checklist, integrity check). Commit.

## TIER 2: Signature features
1. Decision Pipeline animation for every enquiry: Intake, Privacy mask, Understand, Safety check, Route, SLA clock, Draft reply, Human approval gate, each with the real rule/phrase that fired; replay and step-through.
2. Counterfactual explainer: "What would change this result?" re-classifies with each matched phrase removed/negated and shows the effect (for example "without 'chest pain' this becomes: routine").
3. Safety-gated Rules Lab: edit or add phrases/categories/weights in the UI, instantly re-run the whole synthetic dataset in the browser, and show before/after metrics (accuracy, emergency recall, false positives, review rate). Block saving any change that drops emergency recall below 100% and explain why. Export the edited rules.json.
4. Knowledge-base answers for general enquiries: a fictional clinic FAQ (hours, bulk billing, parking, booking, cancellations, telehealth) retrieved with simple TF-IDF/BM25 in plain JS; the draft reply cites which FAQ entry it used and shows the match score; low match goes to human review.
5. Appointment automation: a synthetic appointment calendar with GP/nurse slots; appointment-type enquiries get 2-3 suggested slots in the draft (respecting hours, roles and existing bookings), approval books the slot, and double-booking is prevented and tested.
```

```
6. Complaint and frustration handling: new `complaint_feedback` category plus a simple documented tone/frustration score that raises review priority (never lowers safety priority); escalate to Practice Manager.
7. Language support: detect non-English greeting words and common phrases (a few languages), flag "interpreter may be needed" with the Translating and Interpreting Service number in config, and offer a plain-English reply plus a note; never auto-translate clinical content.
8. Reply studio: tone selector (warm, formal, brief), channel variants (email, SMS with length limit), readability score, and a side-by-side diff between the template and the staff-edited reply. A blocklist test ensures no medical-advice wording. Commit.

## TIER 3: Operations, insight and trust
1. Mission-control dashboard with plain-SVG charts: volume by hour heatmap, priority donut, backlog ageing, SLA compliance gauge, median time to first response, workload by staff, emergency acknowledgement times, human-review rate.
2. Trend and spike detection: a documented z-score/EWMA alert on category volumes, with a "possible cluster of similar symptom messages" signal for manager review only, labelled as a demo signal, never a diagnosis.
3. Clinic-day simulator upgrade: speed control, seeded arrival patterns (morning peak, lunch lull, after-hours trickle, rare emergencies), queue swimlanes with animated cards, live SLA breaches, a staffing slider (receptionists/nurses/GPs) that shows how backlog and SLA change.
4. Safety Net panel: live emergency-recall and false-positive numbers read from the real evaluation output, plus a Red-team console where anyone can paste adversarial messages (negations, typos, injection text, very long input) and see the pipeline result.
5. Tamper-evident audit log shown visually (chain of blocks turning red at the broken link), Verify button, and an Audit Trail view per enquiry.
6. Interoperability (illustrative): export an enquiry/task as FHIR-shaped JSON (Communication and Task resources) and CSV. Label clearly "illustrative, not validated".
7. Printable shift-handover report and installable offline PWA (manifest + service worker, try/catch, works without it).
8. Notifications: optional browser notification and a user-triggered read-aloud for emergency alerts; nothing autoplays. Commit.

## TIER 4: Evaluation, quality and engineering proof
1. Grow the synthetic set to 1,500 enquiries (fixed seed 20261030, labels kept in a separate file never read by triage code) with extra groups: multilingual greetings, complaints, appointment variants, FAQ questions, borderline urgency, injection attempts. Re-run evaluation and write real numbers to `evaluation_metrics.json` and `docs/evaluation_report.md` (accuracy, per-category precision/recall, emergency recall which must stay 100%, false positives, per-edge-group results, confusion matrix, bootstrap 95% CIs, a latency benchmark for classifying the whole set, and a 10-worst-errors analysis with fixes made).
2. Tests: classifier, parity between JS and Python, routing and permissions (enforced by the server API, not just the UI), appointment double-booking, TF-IDF retrieval, rules-lab recall gate, counterfactual explainer, privacy masking, injection cases, hash-chain integrity, SLA and hours logic, reproducibility (same seed gives identical SHA-256, different seed differs). Report real pass counts.
3. Security and accessibility: input limits, output escaping (XSS test), security headers and path-traversal protection in server mode, ARIA live regions for alerts, keyboard operation of everything, contrast check.
4. `.github/workflows/ci.yml` running Node and Python tests; optional Dockerfile; `.gitignore` for runtime files.
5. If a headless browser is available, run an end-to-end pass over the main flows with no console errors, and capture 6-8 real screenshots into `docs/screenshots/` for the README; if unavailable, say so in BUILD_LOG.md. Commit.

## TIER 5: Documentation (no video, no personal reflections)
1. README: a "Try it in 60 seconds" section at the top, feature overview with screenshots, run instructions for static and server modes, GitHub Pages steps, real results table, a table "How this maps to the assessment criteria" (criterion, where it appears, real evidence), limitations and a "Not done yet" list.
2. docs: updated architecture diagrams, safety notes with a hazard log (missed emergency, false reassurance, wrong routing, privacy leakage, hallucination or injection, stale SLA timers, over-trust, wrong language handling), assumptions, and evaluation report. Make no legal or regulatory claims; list open questions a real deployment would need answered.
3. `WHATS_NEW.md` as described above. Do not produce any narration text for the video. Commit.

## PRIORITY IF TIME RUNS SHORT
Tier 1 → Tier 2 items 1, 2, 3, 4, 5 → Tier 4 items 1-2 → Tier 3 items 1, 3, 4, 5 → remaining items → Tier 5. Never leave a feature half-built: finish it or remove it, and list skipped items in README "Not done yet".

## FINAL CHECKS (run, do not assume)
`node --check` all JS, `npm test`, Python tests, evaluation emergency recall = 100%, static mode works from root, subpath and file://, zero `/api/` calls in static mode, no console errors in main flows, no secrets, no local paths, no file over 5 MB, docs match the code.
```

```
## FINAL MESSAGE TO ME
Print: what was added, real test and evaluation numbers, anything skipped, git status, and a numbered
checklist for ME only: run both modes and try everything in WHATS_NEW.md; save this prompt as
`prompt_history/04_max_upgrade_prompt.md` and log my prompts in PROMPTS.md; answer the AI_USAGE reflections
myself; push to my PUBLIC GitHub repo; enable GitHub Pages (Settings, Pages, branch main, root); check the
live URL in incognito; record my own video; email jobs@gokulinfocare.com with repo link, live link, video
link and prompt documentation. End with a verdict: READY AFTER MY TODOs, or the exact blockers.

Begin now with Tier 1 and continue without stopping.
```

Prompt 7 Round 2 prompt (condensed, safe to re-run)

Requested: 30 Oct 2026, 16:31 IST | Tool: AI coding tool (Codex or the second tool)

```
# ROUND 2 UPGRADE PROMPT (condensed, safe to run repeatedly): Clinic Enquiry Desk

## MISSION
Take this hiring-assessment project (a healthcare-technology company; reviewers score practical AI use,
prompt-engineering iteration, creativity, technical execution, usefulness and explainability) from good to
outstanding. Work autonomously, ask no questions, finish within about 90 minutes of work. This prompt may
have been run before: NEVER redo finished work.

## STEP 0: Read the state first (cheap commands only)
1. Read `WHATS_NEW.md`, `BUILD_LOG.md` (tail), `HANDOFF_PROMPT.md` / `HANDOFF_STATE.md` if present; run
`git status`, list files, `node --check` on all JS.
2. Compare what exists with the FEATURE LIST below. Write `ROUND2_STATE.md` with three lists: ALREADY DONE
(verified how), BROKEN (fix first), NOT DONE. Then fix BROKEN items, then build NOT DONE items in the
priority order below. Skip anything already done, and improve it only if clearly weak.

## OWNER BOUNDARIES
- I record the video myself in my own words. Do NOT write or edit any video script or narration, and do not
touch `VIDEO_SCRIPT.md`.
- Do NOT write my reflections in `AI_USAGE.md`, and do not edit my prompt history (`PROMPTS.md`,
`prompt_history/`) except adding this prompt as one factual revision row. Leave all `TODO (owner)`
untouched.
- Keep `WHATS_NEW.md` updated with every new feature and exact click paths (a factual cheat sheet for me,
not a script).

## HARD RULES
- Keep all existing features working. After each tier run `node --check` on all JS, `npm test` and the
Python tests; fix failures. Static mode must work on GitHub Pages (repo root), from a subpath and from
file://, and make no network calls by default.
- Plain HTML/CSS/JS with inline SVG; no CDN, external fonts, frameworks or build step. Python: standard
library + pytest.
- Safety: synthetic data only, no medical advice, human approves every reply, emergency wording always wins
and no feature may downgrade it, no secrets, no real personal data, no local paths. Every number shown is
real or a labelled adjustable assumption; no fake metrics or screenshots. Helpline numbers sit in config
marked "verify before real use".
- Respect `prefers-reduced-motion`, keyboard access, focus and contrast. Log real problems and fixes in
BUILD_LOG.md under "Round 2". Commit after each tier (if git fails, say so once and continue). Be token-
economical: focused edits, no giant outputs. If tokens run low, stop at a stable state and write
`HANDOFF_PROMPT.md`.

## FEATURE LIST (priority order; do only what is missing)
**A. UI/UX polish**
1. Distinctive design system (dark mission-control + light theme toggle), tabular numerals, inline SVG
icons, depth and spacing consistency, icons plus text on every status colour.
2. Sidebar/tab navigation, Ctrl+K command palette, shortcut overlay (?), toasts, undo, skeletons, empty and
error states, dismissible onboarding hints, inline form validation, fully responsive.
3. Hero with live status strip and a 60-second skippable auto-demo with captions (emergency, negated
emergency, look-alike, multi-intent, refill needing GP role and identity checklist, integrity check).

**B. Signature features**
4. Animated Decision Pipeline per enquiry (Intake, Privacy mask, Understand, Safety check, Route, SLA
clock, Draft reply, Human approval) showing the real rule or phrase at each stage, with replay and step-
through.
5. Counterfactual explainer ("what would change this result?" by removing or negating each matched phrase).
6. Safety-gated Rules Lab: edit phrases, weights and categories, re-run the whole synthetic dataset in-
browser, show before/after metrics, block any change that drops emergency recall below 100%, export edited
rules.json.
7. FAQ-grounded replies for general enquiries (fictional clinic FAQ, TF-IDF/BM25 in plain JS) with the
cited FAQ entry and match score; low match goes to human review.
8. Appointment automation: synthetic calendar, 2-3 suggested slots in the draft, booking on approval,
double-booking prevention.
9. Complaint/frustration handling (new category, tone score raises review priority only), language
detection with an "interpreter may be needed" flag, reply studio (tone, email vs SMS, readability score,
template-vs-edited diff, medical-advice blocklist).
```

```
**C. NEW in this round: Trust Center (reviewer-friendly proof inside the app)**
10. A "Trust Center" page with: live in-browser self-tests (classifier cases, hash-chain integrity, recall
gate, routing permissions, double-booking prevention, privacy masking) showing real PASS/FAIL results when
the user clicks Run; the Safety Net panel (emergency recall and false-positive numbers read from the real
evaluation output); the hazard log; and a "Reviewer's 5-minute path" listing which pages to open to verify
each claim.
11. Red-team console: paste adversarial messages (negations, typos, injection text, very long input) and
see the full pipeline result.
12. Tamper-evident log shown visually (chain of blocks turning red at the break), Verify button, per-
enquiry Audit Trail.

**D. Operations and insight**
13. Dashboard in plain SVG: volume-by-hour heatmap, priority donut, backlog ageing, SLA gauge, median time
to first response, workload by staff, emergency acknowledgement times, review rate.
14. Clinic-day simulator: seeded arrival patterns, speed control, swimlanes with animated cards, live SLA
breaches, staffing slider showing the effect on backlog and SLA.
15. Spike/trend alert (z-score or EWMA) on category volume, labelled as a demo signal for manager review
only, never a diagnosis.
16. Illustrative FHIR-shaped JSON export (Communication/Task) plus CSV, clearly labelled not validated;
printable shift-handover report; installable offline PWA (try/catch, works without it); user-triggered
read-aloud and optional browser notifications for emergency alerts (nothing autoplays).

**E. Evaluation, quality, docs**
17. Synthetic set of about 1,500 enquiries (fixed seed 20261030, labels in a separate file triage code
never reads) with groups for multilingual greetings, complaints, appointments, FAQ questions, borderline
urgency, negation, look-alikes, injection. Regenerate `evaluation_metrics.json` and
`docs/evaluation_report.md` with real numbers (accuracy, per-category precision/recall, emergency recall
which must stay 100%, false positives, per-group results, confusion matrix, bootstrap 95% CIs, latency
benchmark, 10 worst errors with fixes).
18. Tests for every feature above (including API-enforced permissions), reproducibility proof (same seed
same SHA-256, different seed differs), security basics in server mode (input limits, escaping with an XSS
test, headers, path-traversal protection), an accessibility pass, `.github/workflows/ci.yml`, `.gitignore`
for runtime files.
19. If a headless browser is available, run an end-to-end pass without console errors and save 6-8 real
screenshots in `docs/screenshots/`; otherwise note it in BUILD_LOG.md.
20. README: "Try it in 60 seconds" at the top, features, run steps for both modes, GitHub Pages steps, real
results table, "How this maps to the assessment criteria" table with evidence, limitations and "Not done
yet". Update architecture, safety notes, hazard log, assumptions (no legal claims).

## PRIORITY IF TIME RUNS SHORT
Fix BROKEN → A → items 4, 5, 6, 10 → 7, 8 → 17-18 → 11-14 → rest. Finish or remove each feature; never
leave one half-built; list skipped items in README "Not done yet".

## FINAL CHECKS (run, do not assume)
`node --check`, `npm test`, Python tests, emergency recall 100%, static mode works from root, subpath and
file://, zero `/api/` calls in static mode, no console errors in the main flows, no secrets or local paths,
no file over 5 MB, docs match the code.

## FINAL MESSAGE TO ME
Print: what changed this round, real test and evaluation numbers, anything skipped, git status, and a
numbered checklist for ME only: run both modes and try everything in WHATS_NEW.md; save this prompt as
`prompt_history/05_round2_prompt.md` and log my prompts in PROMPTS.md; answer the AI_USAGE reflections
myself; push to my PUBLIC GitHub repo; enable GitHub Pages (Settings, Pages, branch main, root); check the
live URL in incognito; record my own video; email jobs@gokulinfocare.com with repo link, live link, video
link and prompt documentation. End with a verdict: READY AFTER MY TODOs, or the exact blockers.

Begin now with Step 0 and continue without stopping.
```

Prompt 8 Final optimisation prompt

Requested: 30 Oct 2026, 17:29 IST | Tool: AI coding tool (Codex or the second tool)

```
# FINAL OPTIMIZATION PROMPT: Clinic Enquiry Desk (upgrade, make submittable, then write my owner guide PDF)

## MISSION
This is the FINAL round before I submit this hiring-assessment project (healthcare-technology company;
reviewers score practical AI use, prompt-engineering iteration, creativity, technical execution, usefulness
and explainability). Work autonomously, ask no questions, and do not redo finished work. Budget: about 75
minutes of building, then the guide. Order of work: (1) read state, (2) fix and finish, (3) upgrade as much
as possible, (4) final audit, (5) write my owner guide PDF last.

## OWNER BOUNDARIES
- I record the video myself in my own words. Do NOT write or edit video narration or a video script, and do
not touch `VIDEO_SCRIPT.md`.
- Do NOT write my reflections in `AI_USAGE.md`, do not invent or edit my prompt history (`PROMPTS.md`,
`prompt_history/`; you may add this prompt as one factual revision row), and leave every `TODO (owner)`
untouched.
- `WHATS_NEW.md` stays a factual cheat sheet (features and click paths), kept up to date.

## HARD RULES
- Keep everything that works. After each step run `node --check` on all JS, `npm test` and the Python tests
and fix failures. Static mode must work on GitHub Pages (repo root), from a subpath and from file://, with
no network calls by default.
- Plain HTML/CSS/JS with inline SVG; no CDN, external fonts, frameworks or build step in the project.
Python: standard library + pytest.
- Safety: synthetic data only, no medical advice, a human approves every reply, emergency wording always
wins and no feature may downgrade it, no secrets, no real personal data, no local paths in tracked files.
Every number shown is real or a clearly labelled adjustable assumption; no fake metrics or screenshots.
Helpline numbers live in config marked "verify before real use".
- Respect `prefers-reduced-motion`, keyboard access, focus and contrast. Use Australian English in all UI
text and docs. Log real problems and fixes in BUILD_LOG.md under "Final round". Commit after each step (if
git fails, say so once and continue). Be token-economical; if tokens run low, stop at a stable state and
write `HANDOFF_PROMPT.md`.

## STEP 0: Read state (cheap commands only)
Read `WHATS_NEW.md`, the tail of `BUILD_LOG.md`, `ROUND2_STATE.md` / `HANDOFF_*.md` if present; run `git
status`, list files, `node --check` everything. Write `FINAL_STATE.md` with ALREADY DONE (verified how),
BROKEN, NOT DONE. Fix BROKEN first.

## STEP 1: Finish the feature set (build only what is missing)
Checklist to compare against:
- UI/UX: dark mission-control + light theme, sidebar/tab navigation, Ctrl+K command palette, shortcut
overlay (?), toasts, undo, skeletons, empty and error states, onboarding hints, inline validation,
responsive layout, hero with live status strip, 60-second skippable auto-demo with captions.
- Core: animated Decision Pipeline with real rule/phrase per stage, counterfactual explainer, safety-gated
Rules Lab (re-runs the dataset in-browser, blocks changes that drop emergency recall below 100%), FAQ-
grounded replies (TF-IDF/BM25 with cited entry and score), appointment suggestions and booking with double-
booking prevention, complaint/frustration handling, language detection with an interpreter flag, reply
studio (tone, email vs SMS, readability, diff, medical-advice blocklist), role routing with API-enforced
permissions, SLA timers, pinned emergency alert needing acknowledgement, sensitive-data masking, identity
checklist, duplicate detection.
- Trust and proof: Trust Center page (in-browser self-tests with real PASS/FAIL, Safety Net panel from the
real evaluation output, hazard log, "Reviewer's 5-minute path"), red-team console, tamper-evident log
visual with Verify button.
- Operations: SVG dashboard (hourly heatmap, priority donut, backlog ageing, SLA gauge, median first
response, workload by staff), clinic-day simulator with staffing slider, spike alert labelled as a demo
signal, illustrative FHIR-shaped JSON and CSV export, printable shift-handover report, installable offline
PWA (try/catch), user-triggered read-aloud.
- Evaluation: about 1,500 synthetic enquiries (fixed seed 20261030, labels in a separate file triage code
never reads), real metrics in `evaluation_metrics.json` and `docs/evaluation_report.md` (emergency recall
must be 100%, per-category precision/recall, per-group results, confusion matrix, bootstrap 95% CIs,
latency, 10 worst errors with fixes), reproducibility proof (same seed same SHA-256, different seed
differs).
Finish or remove each feature; never leave one half-built; list skipped items in README "Not done yet".

## STEP 2: Final polish pass (this is where quality shows)
```

```
1. Design consistency sweep: spacing, typography, colours, button states, hover/focus, icon alignment,
empty states, long-text overflow, tiny-screen layout, light and dark theme parity. Fix every rough edge.
2. Copy sweep: clear plain Australian English, no placeholder or lorem text, no "TODO" visible in the UI,
helpful microcopy on every page, a visible "Demo: synthetic data only, not medical advice" notice.
3. Performance: first load under 1 second locally, classifying the full dataset benchmarked and shown, no
layout jank; avoid unnecessary re-renders.
4. Robustness: empty, huge and script-like inputs, localStorage blocked, offline, reload mid-flow, double-
clicks on Approve, bad imports. Fix and add tests.
5. Accessibility: labels, focus order, ARIA live regions for alerts, contrast, reduced motion.
6. Security basics for server mode: input limits, escaping with an XSS test, headers, path-traversal
protection, safe errors.
7. Repo readiness: `.gitignore` for runtime files, MIT LICENSE, CI workflow (`.github/workflows/ci.yml`),
no file over 5 MB, no secrets, no emails/phones (except public helpline numbers in config), no local paths,
relative asset paths so GitHub Pages works, `.nojekyll`.
8. If a headless browser is available: run an end-to-end pass without console errors and save 6-8 real
screenshots to `docs/screenshots/` for the README; otherwise note it in BUILD_LOG.md.
9. README: "Try it in 60 seconds" at the top, feature overview with screenshots, run steps for static and
server modes, GitHub Pages steps, real results table, a table "How this maps to the assessment criteria"
(criterion, where shown, real evidence), limitations and "Not done yet". Update architecture, safety notes,
hazard log, assumptions, evaluation report. Make no legal or regulatory claims.

## STEP 3: Independent final audit
Audit like a skeptical reviewer with real commands, from a fresh copy in a temp folder: install/run per
README, `node --check`, `npm test`, Python tests, evaluation with emergency recall 100%, reproducibility,
static mode from root, subpath and file://, zero `/api/` calls in static mode, no console errors in main
flows, secrets/personal data/local-path scan, file sizes, docs-match-code check. Fix every failure and re-
run. Write `docs/FINAL_AUDIT.md` (table of checks, PASS/FAIL, evidence, real numbers). Commit.

## STEP 4: Write my owner guide as a single PDF (do this LAST)
Create the folder `owner_guide/` and add it to `.git/info/exclude` so it is never committed to the public
repo. Produce `owner_guide/SUBMISSION_GUIDE.pdf` (about 8-12 pages, clear headings, large readable text,
numbered steps, checkboxes drawn as "[ ]", and copy-paste blocks in monospace). Generate it by writing an
HTML file and printing it to PDF with headless Chrome/Edge if available (no dependency); if not available,
install a PDF library into a temporary virtual environment OUTSIDE the project (never add it to the
project's requirements) and use that; if neither works, produce `SUBMISSION_GUIDE.md` and say so. Verify
the PDF exists, opens, and has the expected page count. The guide must contain, using real file names and
values from this repo:
1. **Where things stand**: what the project is, a plain-English summary of the main features (from
WHATS_NEW.md), the real test and evaluation numbers, and anything skipped.
2. **Schedule to a 9 PM finish**: a time-blocked plan from now (testing, GitHub, Pages, video, documents,
email) with buffers.
3. **Test it yourself in 15 minutes**: exact commands (PowerShell) for both modes, plus a click-through
checklist of every major feature.
4. **Fill-in list**: every `TODO (owner)` in the repo with file and line number, and for each the exact
format to use. For `PROMPTS.md`: the table format with one example row using placeholder values, what
counts as a prompt to log (every prompt sent to any AI tool, plus significant revisions and follow-ups),
where my own prompts to Claude/other tools go, and the rule not to invent anything. For `AI_USAGE.md`: the
reflection questions plus guidance on how to answer them honestly in my own words (what to include, what
not to claim, 3-5 sentences each), without writing the answers.
5. **GitHub steps**: repo name `clinic-enquiry-desk` (Public, no README/licence/gitignore added on GitHub),
a ready description, commands to commit and push (`git branch -M main`, `git remote add origin ...`, `git
push -u origin main`), what to do if git reports ownership/safe.directory or read-only errors, enabling
GitHub Pages (Settings, Pages, Deploy from branch, main, root), adding the live URL in the repo "Website"
box, and the incognito verification checklist for the repo and the live site.
6. **Prompt documentation file**: the assessment wants a document containing all my prompts. Create
`owner_guide/make_prompt_pdf.py` (or an equivalent HTML-to-PDF step) that compiles `PROMPTS.md` and every
file in `prompt_history/` into one `Prompt_Documentation.pdf` WITHOUT editing or inventing any content, and
explain in the guide when to run it (after I have filled my TODOs) and how to check the result. Do not run
it for me with unfilled TODOs.
7. **Video checklist (topics only, no narration)**: the five things the company asked the video to cover
(AI tools used and why; prompt design and refinement; approach and prior knowledge; challenges and how they
were solved; final outcome with a demo), minimum 2 minutes. Add recording options on Windows (Xbox Game Bar
with Win+G, OBS Studio, or a free browser recorder), audio check, a suggested tour order taken from
WHATS_NEW.md, how to export, how to upload to Google Drive or YouTube (unlisted), setting access to "anyone
with the link", and testing it in incognito. Do not write what I should say.
8. **Email**: the To address (jobs@gokulinfocare.com), subject line format ("Task 1 Submission - AI Growth
& Automation Associate - <my name>"), a clean body template with placeholders for the repo link, live link,
video link and prompt-documentation link or attachment, a note on file-size limits and using a Google Drive
link set to anyone with the link, and a final pre-send checklist (links tested in incognito, attachments
open, name and contact details correct).
9. **Do and don't**: be honest about AI use; do not claim to have hand-written AI-generated code; no real
```

```
patient data; keep the repo public until the process is over; do not push large files; do not change
anything after submission unless essential.
10. **After sending**: what to expect (their selection steps: Task 1, then a Python programming assessment,
then HR and final Zoom interviews), a short list of things to be ready to explain about this project
(architecture, classifier rules, safety-first design, evaluation, how AI was used), and a quick-reference
cheat sheet of key numbers and commands.
11. **Troubleshooting**: common problems and fixes (Pages 404, blank page from wrong paths, git read-only
or ownership errors, Codex login problems, large file push rejected, video too large to email).
Make sure the guide contains no secrets, and that nothing in `owner_guide/` is tracked by git (check with
`git status --ignored`).

## FINAL MESSAGE TO ME
Print: what was added or fixed this round, real test and evaluation numbers, anything skipped, git status,
the path of `owner_guide/SUBMISSION_GUIDE.pdf` and `make_prompt_pdf.py`, and a short numbered list of my
immediate next three actions. End with a verdict: READY AFTER MY TODOs, or the exact blockers.

Begin now with Step 0 and continue through Step 4 without stopping.
```

6. Limits of this record

- I did not save the AI tools' responses, only my prompts. I only had the final summary text some tools printed.
- I used two AI coding tools because the first one ran low on its free token allowance. The tool used for each later round is not recorded round by round.
- Almost all of the code was written by the AI tools. I directed the work, ran the prompts, read the results and decided what to change next.
- A few unrelated messages I sent to Claude in the same chat (not about this project) are not listed.
- Claude wrote the prompts from my instructions, so the wording of the prompts is Claude's. The goals, the corrections and the decisions about what to ask for next are mine.